



MASTER'S DEGREE THESIS

Dipartimento di Informatica
Corso di Laurea Magistrale in Informatica

Deep Recurrent Reservoir...etc

Author:

Vincenzo Gargano

Supervisors:

Prof. Claudio Gallicchio

Dott. Andrea Ceni

Dott. Andrea Cossu,

Academic Year 2024/2025

Abstract

...

Contents

Chapter 1

Introduction

1.1 Motivation and Objectives

Nowadays, Artificial Intelligence (AI) became the main driver of innovation in many fields, at the point that the growth of research became exponential in the last decade. This fully aligns with the current state of our world, in fact, we are observing a paradigm shift in the way we interact with technology (or world), through the use of AI.

Within AI, whose main goal is not only to understand *intelligence*, but to embed it into agents capable of performing complex tasks through *rational reasoning*, *planning* and *acting*, the subfield of Machine Learning (ML) has emerged as the dominant approach to mimicking biological intelligent behaviour. Neural Networks (NN) *learn* complex patterns from large amounts of real-world data, instead of relying on explicit rule based approaches, in order to perform *inference* and return prediction on unseen data.

The last years have seen a surge in popularity of generative models, after a first wave of discriminative models that made popular the field of Deep Learning (DL), and introduced GPU computation in 2009 [1]. Particularly, deep generative models have been applied with unbelievable results in many fields, from image generation with *Diffusion Models* [2] to text generation with *Transformers* [3] and protein folding with *AlphaFold* [4] and many others domains, achieving state of the art results. However, deep learning models are known to be expensive in both training and inference, requiring large amount of computational resources (GPU production) and energy consumption [5].

Forse dovrei concentrarmi più su RNN dall'inizio?

1.2 Reservoir Computing for Efficient AI

Sequential data processing remains a fundamental challenge in machine learning, from natural language to time series forecasting. While Recurrent Neural Networks (RNNs) have been the traditional approach for modeling temporal dependencies, training them to capture long-term patterns has proven notoriously difficult due to vanishing and exploding gradients. Although architectures like LSTMs and GRUs

mancano citazioni e è ripetitivo rispetto al background

have partially addressed these issues through gating mechanisms, and Transformers have recently dominated the field with their attention-based approach, these solutions come at the cost of increased architectural complexity and computational demands.

An alternative paradigm that has gained attention is Reservoir Computing (RC)[1], which offers a fundamentally different approach to sequential processing. By maintaining a fixed, randomly initialized recurrent network as a "reservoir" of dynamics, and training only a simple linear readout layer, RC circumvents the gradient-based training of recurrent connections entirely. This framework not only simplifies the learning process but also draws inspiration from neuroscience and physical computing systems, where rich dynamics emerge naturally without explicit optimization.

Despite its computational efficiency and theoretical elegance, vanilla RC models face their own set of limitations, particularly in scaling to complex tasks that require multiple levels of temporal abstraction[2]. This motivates the exploration of *deep* variants [3] of reservoir computing architectures, which aim to combine the efficiency of the RC paradigm with the representational power of hierarchical processing. This thesis investigates such deep reservoir architectures, examining how stacking multiple reservoir layers can enable learning at different timescales while maintaining the training efficiency that makes RC attractive.

After finishing experimental part...

1.3 Scientific contribution

1.4 Structure of the work

elenco puntato con quello che si dirà in ogni capitolo della tesi

Chapter 2

Background

In this chapter we will review the main concepts and theory needed to understand the rest of this thesis. We will start with a brief introduction of the Dynamical Systems (DS) theory and an overview of Machine Learning concepts and foundational models that will be useful to understand the rest of the work.

2.1 The Inherent Dynamism of Natural Systems

Our universe is essentially a complex non-linear dynamical system, and since Newton in mid 1600s, used differential equations as a method to solve the two body problems, but many natural phenomena are essentially Complex Systems, from small components of the environment there are interdependences (CS) which exhibit dynamics that evolve through time: such as motion of planets, flow of fluids, weather changes, growth of population and many others phenomena.

Dynamics is a fundamental piece of reality and modeling time dependent system evolution, is crucial to understand the world around us: eg. the design of control system, stock market and also the human brain can be modeled as a dynamical system. The methodological framework developed through the preceding century have been specifically designed to analyze sequential data structures, commonly referred as *time series*, These approaches encompass techniques such as Autoregressive (AR) models (i should cite but cannot find source) which employ linear regression on data sequences, extending to more sophisticated frameworks including State-Space Models [6], originally developed for Control Theory applications, and Hidden Markov Models (HMM) [7]. The latter served as foundational precursors to Recurrent Neural Networks [8] (RNNs). The aforementioned models, are the basis of modern neural based sequential modeling.

questo va nel background

devo approfondire i sistemi complessi?

forse questa parte s framework non va messa qui

2.2 Dynamical Systems review

A dynamical system is a system which evolves over time, we can describe it's changes through **state variables**, usually many models are constructed such that only the fundamental state variables are considered, for example *biological spiking neurons* modeled as in [9] are described by a simple 2-D dimensional ODE, where the two state variables are the membrane potential and a recovery variable. ignoring existing other possible variables such gradient of ion concentration, temperature; for the sake of obtaining a large scale simulation.

There are two types of dynamical systems: *Differential Equations*, which are continuous in time, and *Difference Equations* or *Iterated Maps* which are discrete in time. The most used in a wide range of sciences fields are the former, which are further distinguished in Ordinary Differential Equations (ODE) and Partial Differential Equations (PDE), the difference is that in PDE we have two or more independent variables.

This thesis will focus on ODEs since our models fell into that category, but we will also use *discretization methods* (link to the section maybe) to obtain it's approximated solution.

Formally we define an ODE as:

$$\frac{d\mathbf{x}}{dt} = f(\mathbf{x}, t) \quad (2.1)$$

where $\mathbf{x} \in \mathbb{R}^n$ is the state vector, t is the time and $f : \mathbb{R}^n \times \mathbb{R} \rightarrow \mathbb{R}^n$ is a vector field grouping all the state variables that describe the evolution of the system.

An example of ODE is the well known Dampened Harmonic Oscillator:

$$\frac{d^2x}{dt^2} + 2\beta \frac{dx}{dt} + \omega_0^2 x = 0 \quad (2.2)$$

where x is the position of the oscillator, β is the damping coefficient and ω_0 is the natural frequency of the oscillator.

A Differential Equation is *autonomous* if the function f does not depend explicitly on time, otherwise is *non-autonomous*. We can switch from a non-autonomous to an autonomous system by adding time as a new variable to the state vector, for example the non-autonomous system of equation ?? can be rewritten with a change of variable.

questo va tolto, forse
meglio non andare
oppo in dettaglio
in queste cose

anche questo va
tolto?

2.2.1 Modeling

Modeling is a central methodology in the study of complex systems, translating real world observations and hypotheses into a formalized mathematical model, even though simplified, can be crucial for studying emergent behaviours, particularly in natural systems where we cannot change the initial state, or the environment variables that influence it.

The Hodgkin-Huxley model [10] describes the dynamics of neuronal membrane potential through an equivalent electrical circuit. The membrane can be represented as a capacitor C_m in parallel with several ion channels, each modeled as a voltage-dependent conductance in series with a battery representing the ion's reversal potential.

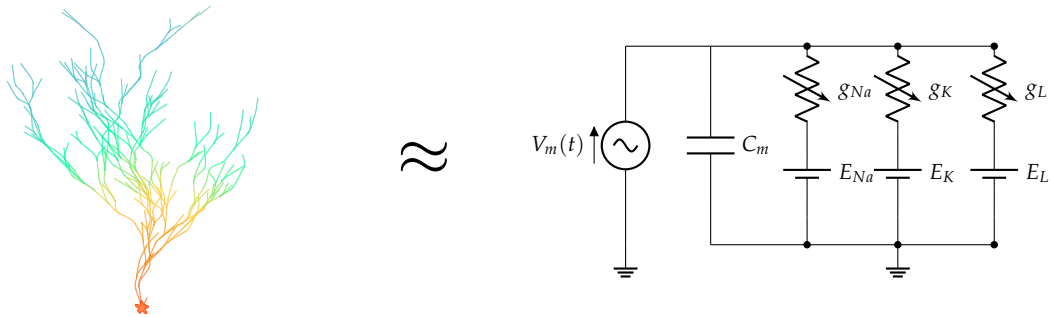


FIGURE 2.1: Biological neuron and its Hodgkin-Huxley equivalent circuit representation.

The corresponding differential equation describing the membrane potential dynamics is:

$$C_m \frac{dV_m}{dt} = I_{ext} - g_{Na}(V_m - E_{Na}) - g_K(V_m - E_K) - g_L(V_m - E_L) \quad (2.3)$$

where I_{ext} is the external input current, and each conductance g_i is voltage and time dependent, governed by additional gating variables.

However, as previously mentioned, natural systems are often complex, as in this case we have a four dimensional non-linear coupled system, accurate modeling can be computationally infeasible, thus often simplified models are preferred for large scale simulations, such as the Izhikevich neuron model [9] that captures essential spiking dynamics with only two state variables.

2.2.2 Discretization

quanto in dettaglio?

2.3 Overview of Machine Learning

Machine Learning (ML) is a subfield of Artificial Intelligence (AI) that focuses on statistical methods and algorithms capable to *learn* from past experiences. First we need a rigorous definition of what means to learn; Mitchell [11] defines it as : "A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E ."

ML can tackle a much wider range of tasks that are difficult to solve with explicit rules based programming, in a certain sense it bypasses some level of abstraction, we're not learning the task T directly, but we're sampling from the E space, **examples** which are collection of **features** that are qualitative or quantitative properties of the data that are exploited by the model to learn the task T .

2.3.1 The learning problem

A formal definition of a machine learning model is given below:

Definition 1. Let $p_{data}(\mathbf{x})$ be an independent identically distributed (i.i.d.) unknown probability distribution over the input space $\mathcal{X}$. A dataset is a finite sample drawn from this distribution, $\mathcal{D}_{dev} = \{\mathbf{x}_i\}_{i=1}^N \subset \mathcal{X}$, where $\mathbf{x}_i \in \mathbb{R}^d$ are data points (or input features).

A **machine learning model** is a family of prediction functions $f : \mathcal{X} \rightarrow \mathcal{Y}$ parameterized by θ . The nature of the model and its goal is defined by the learning paradigm:

- In the **supervised** setting, the data is drawn from a joint distribution $p_{data}(\mathbf{x}, y)$, so $\mathcal{D}_{train} = \{(\mathbf{x}_i, y_i)\}_{i=1}^k$, with targets $y_i \in \mathcal{Y}$. Here, $\mathcal{Y}$ (e.g., $\mathbb{R}$ for regression or $\{0, 1\}$ for classification), and the goal is to find parameters θ^* that minimize a loss function $\mathcal{L}(y, f(\mathbf{x}; \theta))$ quantifying the discrepancy between predictions and true labels.
- In the **unsupervised** setting, the data consists only of inputs $\mathbf{x}_i$. The output space $\mathcal{Y}$ is defined by the specific task (e.g., $\mathbb{R}^k$ for dimensionality reduction, a set of clusters for clustering, or $\mathbb{R}^d$ for generative models). The goal is to find parameters θ^* that optimize an objective function $\mathcal{J}(f(\mathbf{x}; \theta), \mathcal{D})$, which captures desired structural properties of the data, such as reconstruction error, cluster cohesion, or data likelihood.

In both cases, the ultimate objective is for the model $f(\mathbf{x}; \theta^*)$ to generalize well to unseen data $\mathcal{D}_{test} \sim P(\mathbf{x})$.

The model f_θ ¹ is said to belong to a **hypothesis space** $\mathcal{H}$, which is the set of all possible functions the model can represent by varying its parameters θ . The choice of model architecture (e.g., linear models, neural networks, decision trees) defines the

¹We will use f_θ as shorthand for $f(\mathbf{x}; \theta)$

hypothesis space. A larger, more expressive $\mathcal{H}$ can represent more complex functions. The learning algorithm searches within $\mathcal{H}$ to find the hypothesis $f^* = f(\cdot; \theta^*)$ that best approximates the true underlying relationship between inputs and outputs.

2.3.2 Learning Paradigms

Models learn the data distributions in three main different styles:

1. **Supervised Learning**, where we aim for accurate predictions.
2. **Unsupervised Learning**, tries to find a compact description of the data
3. **Reinforcement Learning**, where an agent recursively interacts with an environment in order to maximize a cumulative reward signal, through a certain policy: π^* .

Understanding learning is crucial to choose the right modeling technique and architecture for a given task. In order to have a more compact picture of learning let's decompose the more general distribution: $P(\mathbf{X}, Y)$

Generative and Discriminative Learning (Bayesian view)

The joint distribution $P(\mathbf{X}, Y)$ decomposes as:

$$P(\mathbf{X}, Y) = P(Y|\mathbf{X})P(\mathbf{X}) = P(\mathbf{X}|Y)P(Y) \quad (2.4)$$

$$\text{Joint} = (\text{Posterior}) \cdot (\text{Marginal}) = (\text{Likelihood}) \cdot (\text{Prior})$$

In machine learning, the primary goal is often to predict the label Y for a given data-point $\mathbf{X}$, this is represented by the *posterior distribution*. One could also be interested in approximating the unknown distribution itself to then generate new samples from it, in that case, are needed *likelihood* and the *prior* distributions.

Bayesian Decision Theory [12] provides a statistical framework for the design of many machine learning algorithms. The way model parameters are estimated often reflects whether the approach is *Generative* or *Discriminative*.

² Discriminative models rely often on conditional distribution, the posterior, and whenever we can incorporate prior knowledge on the parameters, inference is performed through **Maximum A Posteriori** (MAP) estimation^{??}, which seeks the best hypothesis (θ) given the data and prior knowledge.

²However, this correspondence is not absolute [12, p. 312, Ch. 13]: logistic regression can also be trained by pure MLE, and generative models can incorporate priors and thus be estimated via MAP.

c'è ne sono molti altri: semi-supervised, active learning etc.,

Vorrei evitare di spingere Bayes, ma come introduco questi concetti?

forse è meglio ridurre il numero di formule e tenerle più discorsive?

$$\theta_{\text{MAP}}^* = \arg \max_{\theta} P(\theta | \mathcal{D}) = \arg \max_{\theta} P(\mathcal{D} | \theta) P(\theta) \quad (2.5)$$

On the other hand generative models learn the joint distribution, and are typically trained by a **Maximum Likelihood Estimation** (MLE) ??, fitting the parameters that maximize the likelihood of the data.

$$\theta_{\text{MLE}}^* = \arg \max_{\theta} P(\mathcal{D} | \theta) \quad (2.6)$$

2.3.3 Risk, Generalization and Overfitting o (Empirical view)

While the Bayesian perspective ³ provides a probabilistic view for learning, in practice what is often adopted is called the *empirical* approach based on risk minimization [13]. This complements it by focusing on measuring and optimizing performance through the loss function, with the main goal of achieving *generalization*. We seek a function f_{θ} that minimizes the expected risk $\mathcal{R}(f_{\theta})$ over $p_{\text{data}}(\mathbf{x}, y)$. The loss function acts as a proxy for the learning process, risk have a dual nature: the expected risk and the empirical risk.

The **expected risk** (or population risk) of a hypothesis f_{θ} measures its average performance over the true data distribution. However, we do not have access to the entire p_{data} we can only observe a finite sample $\mathcal{D}_{\text{train}}$, therefore, we approximate the expected risk with the **empirical risk**: $\hat{\mathcal{R}}(f_{\theta})$, which is the average loss over the training set.

The learning problem becomes finding θ^* that minimizes $\hat{\mathcal{R}}(f_{\theta})$ over the training set. This is the **Empirical Risk Minimization** (ERM) principle, which forms the foundation of most practical machine learning algorithms.

Risk Decomposition

Most of the times the space of possible solution is unfathomably large, thus we need to decompose the risk in order to understand if we are generalizing correctly. Let $f_{\mathcal{H}}^* = \arg \min_{f \in \mathcal{H}} \mathcal{R}(f)$ be the best possible function within the hypothesis space, and let $\hat{f}_{\mathcal{H}} = \arg \min_{f \in \mathcal{H}} \hat{\mathcal{R}}(f)$ denote the empirical risk minimizer found by the learning algorithm. Then, the population risk of the learned model can be decomposed in: [14][13][15][16, pag. 33, Ch 2]

$$\mathcal{R}(f_{\theta}) = \underbrace{\mathcal{R}(f_{\theta}) - \hat{\mathcal{R}}(f_{\theta})}_{\text{Generalization error}} + \underbrace{\hat{\mathcal{R}}(f_{\theta}) - \hat{\mathcal{R}}(f_{\mathcal{H}}^*)}_{\text{Optimization error}} + \underbrace{\hat{\mathcal{R}}(\hat{f}_{\mathcal{H}}) - \mathcal{R}(f_{\mathcal{H}}^*)}_{\text{Estimation error}} + \underbrace{\mathcal{R}(f_{\mathcal{H}}^*)}_{\text{Approximation error}} \quad (2.7)$$

³As said in [12, p. 314, Ch. 13], Bayesian approach appears to be formally optimal, but a pragmatic approach is possible, it can be conditioned by a penalty λ on the complexity of the model

The approximation error is irreducible because the data-generating process includes an intrinsic noise ε that cannot be captured by any hypothesis in $\mathcal{H}$. Consequently, learning algorithms must focus on reducing the remaining components. When the selected hypothesis f_θ is insufficiently expressive, **underfitting** occurs: the empirical risk $\hat{\mathcal{R}}(f_\theta)$ may be small, yet the estimated hypothesis fails to approximate the target function.⁴ Conversely, if f_θ possesses excessive capacity, the estimator can *interpolate* $\mathcal{D}_{train}$, attaining negligible empirical risk, but with a large generalization gap; this phenomenon is called **overfitting**.

2.3.4 Optimizing and Regularizing

We extensively talked about different ways of learning, but yet we haven't discussed on how the learning is actually performed. Many of the modern ML algorithm are based on **Gradient-Based Optimization** methods [17], which iteratively update the model parameters θ in order to minimize the loss function $\mathcal{L}(y, f(\mathbf{x}; \theta))$. However learning differs from pure optimization, ML methods optimize indirectly with respect to the population risk. The typical ML objective function minimized is:

$$\mathcal{L}(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}_{train}} [\mathcal{L}(y, f(\mathbf{x}; \theta))] + \lambda \Omega(\mathbf{w}) \quad (2.8)$$

The term $\Omega(\mathbf{w})$ is a regularization term that allows to control the complexity of the model, λ is an hyperparameter that controls the strength of the regularization.

Regularization can be done in different ways for example, using a prior distribution over the parameters, or in optimization we can penalize with different norms, for example $L2$ or $L1$ norms, if we are interested in decreasing magnitude of parameters or inducing sparsity respectively.

The algorithm that empowers the entire ML is **Stochastic Gradient Descent** (SGD) [18, pag. 152, Ch 5.2] which iteratively updates the parameters θ in the opposite direction of the gradient of the loss function with respect to the parameters. Typically the parameters are initialized in some configuration θ_0 , a good initialization is often crucial to convergence, and at each iteration t we compute the gradient of the loss function with respect to the parameters $\nabla_\theta \mathcal{L}(\theta_t)$, then we update the parameters using with a learning rate η that controls the step size or magnitude of update and usually is one the most influential factors that determines how quickly and accurately the model converge in a region with an optimal solution.

The learning rule is written as:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t) \quad (2.9)$$

More advanced optimization algorithms exist, such as **Adam** [19] but are based on SGD with some modifications. Moreover SGD can work online or in batch mode, meaning that we can apply the learning rule ?? for each $x_i \sim \mathcal{D}_{train}$ or, the first is

⁴In the case of limited data availability

questa è una parte più pragmatica che spiega come si fa learning e come si gestisce la complessità

comunque nella tesi non facciamo molto uso di gradient based optimization, forse si può sovolare sui dettagli o serve comunque come background?

preferred when $|\mathcal{D}_{train}|$ is large, and the second when is small enough to fit in memory or if we want the optimization to be noisy, favouring possibility to escape local minima.

Model selection and Hyperparameters

The standard pipeline for training a machine learning model follows a systematic approach. Given a specific task and a sampled dataset $\mathcal{D}_{dev} \sim p_{data}$ partitioned into three subsets: training $\mathcal{D}_{train}$, validation $\mathcal{D}_{val}$, and test sets $\mathcal{D}_{test}$. The training set optimizes model parameters, the validation set serves for hyperparameter tuning and architectural decisions, and the test set provides an unbiased performance estimate. Hyperparameters are those parameters not learned during training but set prior to the learning process. We have seen examples like the regularization coefficient λ and the learning rate η . The training phase primarily reduces the estimation error $\mathcal{R}(f_\theta) - \hat{\mathcal{R}}(f_\theta)$, from eq. ???. Model selection, conversely, controls the approximation error $\mathcal{R}(f_\theta^*)$ by determining the expressiveness of $\mathcal{H}$ through hyperparameter choices. Model selection involves training multiple candidate models with varying architectures and hyperparameters on the training set, then selecting the configuration with the best validation performance. Validation methods, such as K-fold cross-validation [] or Bootstrapping [] and Bayesian optimization [] are often used in order to search more efficiently in the space of possible models.

orse devo prima sp-
gare cos'è unbiased
timate?

2.4 Neural Networks and Deep Learning

Neural networks (NNs) [20, Ch. 3.1] are flexible non-linear universal function approximators [21] [16] which are inspired by biological neural system, where information is processed in hierarchical manner, in fact, the idea originates from the layerwise brain organization in the visual cortex where information is processed at different complexity scales as discussed in [22]. Specifically a shallow feedforward neural network (FFN) is composed of interconnected nodes called *perceptrons* [23] or artificial neurons divided in: input layer, that receives the perceived signal $\mathbf{x} \in \mathbb{R}^d$, one hidden layer, and an output layer that predicts the target $\hat{y}$. By stacking hidden layers we are composing together many different functions eg. $f_{net}^l \circ f_{net}^{l-1} \circ \dots \circ f_{net}^1$, with each hidden layer having its own parameters: $\theta = \{W^{(l)}, \mathbf{b}^{(l)}\}$, increasing the complexity of the model as well the hypothesis space $\mathcal{H}$.

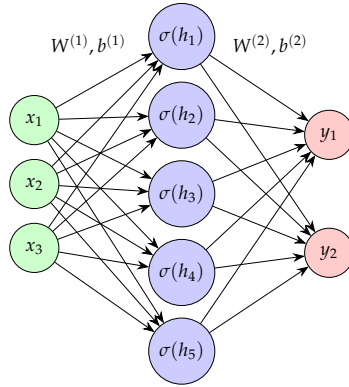


FIGURE 2.2: A simple Feedforward Neural Network with one hidden layer.

This is the main idea behind **Deep Learning** [20, Ch 4.3], which refers to Deep neural networks (DNNs) which are the core of many state-of-the-art machine learning systems [24].

Consider f_{net}^l to be the generic output at layer l of a DNN, composed by two parts: an affine transformation W , followed by a non-linear activation or transfer function σ .

$$f_{net}^l(\mathbf{x}) = \sigma(W^{(l)}\mathbf{x} + b^{(l)}) = \sigma\left(\sum_{i=1}^d w_i^{(l)} x_i + b^{(l)}\right)^5 \quad (2.10)$$

We call $W^{(l)}$ the weight matrix, $\mathbf{b}^{(l)}$ the bias vector, σ is an arbitrary⁶ non linear function such as ReLU, Sigmoid or Tanh. $\mathbf{x} \in \mathbb{R}^d$ is the input vector. The parameters of this model, weight are estimated trough an algorithm known as **Backpropagation**

⁵With x_i and $w_i^{(l)}$ the i -th element of $\mathbf{x}$ and $W^{(l)}$ respectively

⁶While for hidden layers the right activation is usually chosen by model selection, for output layer the activation regards the task e.g., identity for regression, logistic for binary classification and softmax for multiclassifiers

[25], which is an application of the chain rule of calculus to compute the gradient of the loss function with respect to the model parameters.

2.4.1 Recurrent Neural Networks for Sequential Data

Recurrent Neural Networks (RNNs) [25] are a class of neural networks designed to handle sequential values. Their recurrent architecture introduces an *architectural bias* [26] towards capturing sequential dependencies. Even prior to training, recurrent units can display clustering resembling markovian dynamics.

In contrast FFNs, process inputs independently, requiring fixed length data with separated parameters for each input dimension, also sequential data exhibit temporal dependencies and long context which complexity grows with respect to number of timesteps T .

Formally, RNNs model the joint distribution of a sequence $\mathbf{x}_{1:T} = [\mathbf{x}_1, \dots, \mathbf{x}_T]$, by factorizing it over T steps as a product of conditional distributions [27]:

$$p(\mathbf{x}_{1:T}) = \prod_{t=1}^T p(\mathbf{x}_t \mid \mathbf{x}_{1:t-1}; \theta) \quad (2.11)$$

RNNs introduce parameter sharing across time steps, allowing them to generalize across different sequence lengths and capture temporal dependencies. Parameters sharing embodies a key *inductive bias*: the assumption of temporal stationarity. This assumption implies that the same parameters are updated each time step, so that the conditional probability distribution over variables at time $t + 1$ given all previous time steps do not depend explicitly on t , but only on the hidden state at time t .

For any time indices $t, k \in \mathbb{N}$, transition and output distributions: $p(\mathbf{h}_{t+1} \mid \mathbf{h}_t; \theta) = p(\mathbf{h}_{k+1} \mid \mathbf{h}_k; \theta)$ and $p(\mathbf{y}_t \mid \mathbf{h}_t; \theta) = p(\mathbf{y}_k \mid \mathbf{h}_k; \theta)$, share the same parameter set θ governing the dynamics at each time step, consequently, temporal dependencies are captured solely through the evolution of the hidden state $\mathbf{h}_t$, without explicit dependence on t .

Similarly to seminal works as Time Delay Neural Networks (TDNNs) [waibel1989phoneme], RNNs maintain a hidden state $\mathbf{h}_t$ that is updated at each time step based on the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$.

Computational graphs are often used to represent the structure of RNNs, this is true because they are part of the family of structured probabilistic models or **Graphical Models** [12], i.e., Hidden Markov Models (HMMs), Bayesian Networks (BNs) and Conditional Random Fields (CRFs). Where nodes represent random variables and edges represent dependencies between them, similarly we can think to RNNs as a specialization of these probabilistic models, but with a parameterized approach on the dependencies.

RNNs can be *unrolled* through time, which results in a directed acyclic graph (DAG),

this helps visualizing the flow of information trough such networks, as we can see from the figure ??.

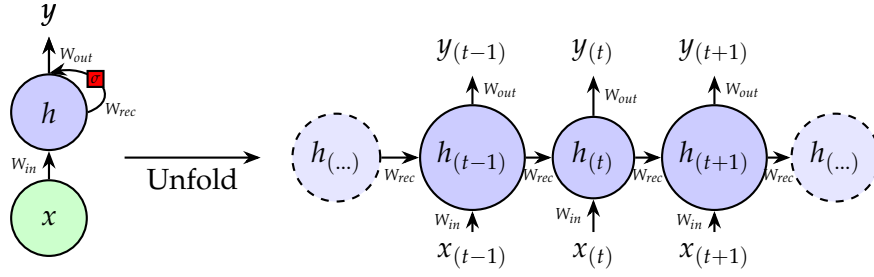


FIGURE 2.3: Recurrent Neural Network unrolled through time

RNNs differs from traditional DNNs, due to the sequential input nature, thus even training is performed in a different way, the algorithm used for training these models is called **Backpropagation Through Time** (BPTT) [28], which requires to backpropagate the error trough from the end to the start of the unrolled graph ?? with $O(T)$ complexity.

As proved by Funashi and Nakamura [29], Continuous Time Recurrent Neural Networks (CTRNNs) are able to approximate finite time dynamical systems with arbitrary precision. We derive a discretized version of equation ?? as follows

$$\mathbf{h}(t) = \tanh(W_{in}\mathbf{x}(t) + W_{rec}\mathbf{h}(t-1) + \mathbf{b}) \quad (2.12)$$

At each time step t , the hidden state $\mathbf{h}(t)$ is updated based on the current input $\mathbf{x}(t)$ and the previous hidden state $\mathbf{h}(t-1)$: Where W_{in} is the input-to-hidden weight matrix, W_{rec} is the hidden-to-hidden (recurrent) weight matrix, $\mathbf{b}$ is the bias vector, and σ is a non-linear activation function such as Tanh or ReLU. The output $\hat{y}^t$ at each time step can be computed as:

$$\hat{y}(t) = W_{out}\mathbf{h}(t) + \mathbf{c} \quad (2.13)$$

Where W_{out} is the hidden-to-output weight matrix and $\mathbf{c}$ is the output bias vector. Finally we get the following DE for a general RNN:

$$\frac{d\mathbf{h}(t)}{dt} = -\mathbf{h}(t) + \tanh(W_{in}\mathbf{x}(t) + W_{rec}\mathbf{h}(t) + \mathbf{b}) \quad (2.14)$$

2.4.2 The challenge for Long-Term Dependencies

RNNs are powerful models for sequential data modeling, however in the last years, they have been largely replaced by Transformer architectures [3], this happened mainly because of the difficulty in training RNNs to capture long-term dependencies in sequences [5], [30], [31]. The problem is addressed as *information latching* [32], which consist in the ability of such network to retain a robust representation of bits

forse questa parte
non va bene non
c'entra molto

of information, this happens when the states of the network are contained in a reduced attracting set of an hyperbolic attractor [33]. Core of this work as well is the investigation of this problem, thus we will briefly summarize the main issues and solution proposed in literature.

By looking again at picture ??, if we imagine a really long sequence, the gradients will be propagated back through many time steps, this can lead to numerical instability and make long term dependencies hard to learn. This is due to the repeated multiplication of the gradients by the Jacobian matrix of the hidden state with respect to itself at the previous time step. Essentially we compute the following loss:

$$\frac{\partial \mathcal{L}_t}{\partial W_{rec}} = \sum_{k=1}^t \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \left(\prod_{l=k}^{t-1} \frac{\partial \mathbf{h}_{l+1}}{\partial \mathbf{h}_l} \right) \frac{\partial \mathbf{h}_k}{\partial W_{rec}} \quad (2.15)$$

where $\frac{\partial \mathbf{h}_{l+1}}{\partial \mathbf{h}_l}$ is the Jacobian matrix at time step l . The product $\prod_{l=k}^{t-1} \frac{\partial \mathbf{h}_{l+1}}{\partial \mathbf{h}_l}$ grows exponentially with the sequence length $(t - k)$, leading to *vanishing* or *exploding* (V/E) gradients ⁷ [24, pag. 290, Ch 8.2] depending on the spectral properties of these Jacobian matrices. Suppose that W_{rec} eigendecomposes as $W_{rec}^t = V \text{diag}(\lambda)^t V^{-1}$. Exploding happen when the $|\lambda_{max}| > 1$, while vanishing when $|\lambda_{max}| < 1$. Essentially the jacobian matrix provide information about how small changes in h_t propagates to the next step or equivalently how the gradient on h_{t+1} propagates at the previous step, moreover the recurrent matrix and the jacobian can have complex-valued eigenvalues, with imaginary part that induces oscillatory dynamics, making more complicated dynamics easier to learn.

Several solutions have been proposed in literature to mitigate this problem, such as gradient clipping [34], residual connections [35] that help by adding to the timesteps directly the signal from previous steps, as well approaches directly acting on the recurrent matrix that fixes its spectrum to 1, such as orthogonal or unitary RNNs [arjovsky2016unitaryevolutionrecurrentneural] [36]. Even choosing the right transfer function can help: Sigmoidal activations tend to saturate very quickly since their derivatives approaches 0 in the extreme regions, alternatives such as tanh that saturates less, or ReLU that has more stable gradients and regularize at the same time by sparsifying activations [37] contribute to the architectural bias.

A fundamental insight into solving the vanishing gradient came from Hochreiter et al. [5], introducing the concept of **Constant Error Carousel** (CEC), which allows gradients to flow unchanged trough time in the backward pass by using self-loops with unit weight in the recurrent connections. But with CEC the error is integrated over time, and time based tasks often require memory to change dynamically.

Gated RNNs

To address the memory issues of learning long-term dependencies in sequential data, specialized RNN architectures with gating mechanisms have been developed,

⁷Gradient issues is related to depth of a network, even sufficiently deep FFNs suffers of V/E issues.

specifically Long Short-Term Memory (LSTM) networks [5] and Gated Recurrent Units (GRUs) [cho2014learningphraserepresentations]. These gates in these models solve the CEC problem by allowing the network to learn selectively what information should be integrated or forgotten over time.

The *forget gate* uses sigmoid activation to decide what amount of information discard from the previous h_t , while the *input gate* control how much of the new information from the current input x_t should be added to the cell state. Finally the *output gate* determines what part of the cell state should be output as the new hidden state h_t .

2.5 Reservoir Computing

Reservoir Computing (RC) is a computational framework introduced in the early 2000s with two keys models: Echo State Networks (ESN) [38] and Liquid State Machines (LSM) [39]. Both approaches aims to simplify the training of RNNs while exploiting the inherent dynamical properties of real world physical systems to obtain a "reservoir" of rich dynamics that can be leveraged to solve complex task. A major advantage of RC lies in it's efficiency: only the linear readout layer is trained, while recurrent connections remains fixed.

The central idea is to avoid training the recurrent connection in the network by using a fixed high-dimensional reservoir, that project the input signal into an high dimensional feature space, where a linear classifier is sufficient to learn the linearized representation of the input signal.

Reservoir Computing has been successfully applied to tasks such as time series prediction, speech recognition, robot control , furthermore can also be extended to generative settings using techniques such as Teacher Forcing [40].

add citation here

The focus of this work is on RC methods that address the challenge of solving learning long-term dependancies, by directly avoid the use of BPTT and its issues. From a theoretical viewpoint, RC models can be viewed as generalization of **kernel methods** to sequential data, these type of models solve the task in two steps: firstly they project the input data into a high dimensional space using a fixed non-linear mapping (the reservoir in RC), secondly they learn a linear model on top of this representation to make predictions.

Both RC and kernel methods relies on the **Cover's Theorem** [41], which states :

"A complex pattern-classification problem, cast in a high-dimensional space nonlinearly, is more likely to be linearly separable than in a low-dimensional space."

2.5.1 Echo State Networks

ESNs are a specific type of fully connected RNN, where only a small part of recurrent connection in the hidden layer is active, inducing sparsity, and only the output

weights are trained. A common approach to build rich reservoirs [24][Ch 10.8] is to design the recurrence within a state-space system that operates near the *edge of chaos* regime as described by [42] [43]. The idea is to constrain the eigenvalues of the system's Jacobian matrix $\mathbf{J}$ to be close to 1. As we previously mentioned in sec ??, we want to enforce this condition on: $\mathbf{J}_t = \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$.

In practice, this is controlled by the **spectral radius** of such $\mathbf{J}$, defined as the largest absolute eigenvalue of the matrix, denoted as $\rho(\mathbf{J}) = \max |\lambda_i|$. By constraining $\rho(\mathbf{J})$ to be close to 1, we can ensure that the system is in the edge of chaos regime, which is characterized by a balance between stability and instability.

In ESNs we can control the spectral radius of the recurrent weight matrix W_{rec} directly during initialization by scaling it appropriately. Given a randomly initialized weight matrix W_{rec}^{init} , we can compute its spectral radius $\rho(W_{rec}^{init})$ and then scale it to a desired value $\rho_{desired}$:

$$W_{rec} = \frac{\rho_{desired}}{\rho(W_{rec}^{init})} W_{rec}^{init} \quad (2.16)$$

Once the reservoir is initialized, we can use it to process input sequences and collect the reservoir states over time. The readout layer is then trained using closed form solutions as ridge regression or pseudoinverse.

A more general formulation is the Leaky ESN is given by [44], which introduces a leaking rate $\alpha \in [0, 1]$ to control the speed of the reservoir dynamics:

$$\mathbf{h}(t) = (1 - \alpha)\mathbf{h}(t - 1) + \alpha \tanh(W_{in}\mathbf{x}(t) + W_{rec}\mathbf{h}(t - 1) + \mathbf{b}) \quad (2.17)$$

When $\alpha = 1$, the Leaky ESN reduces to the standard ESN, while for $\alpha < 1$, the reservoir state updates more slowly, allowing it to capture longer-term dependencies in the input sequence.

Echo State Property

The entire field of RC, particularly ESNs, relies onto the assumption that the reservoir's dynamics are stable and that the current state is uniquely determined by the input history. The **Echo State Property** (ESP) [38] formalizes this requirement, stating that for any bounded input sequence, the reservoir state should asymptotically depend only on the input history and not on the initial state.

Formally, let $\mathbf{x}(t)$ be a bounded input sequence and $\mathbf{h}_1(0)$, $\mathbf{h}_2(0)$ be two different initial reservoir states obtained by running that sequence through the reservoir. The ESP requires that:

$$\lim_{t \rightarrow \infty} \|\mathbf{h}_1(t) - \mathbf{h}_2(t)\| \rightarrow 0 \quad (2.18)$$

When this condition holds, the model behaves as a *fading memory* filter, where the influence of past inputs diminishes over time, in DS theory these are called

also *contractive systems* [addcitation]. For it to be true, we must impose two conditions, a necessary and a sufficient one, both regarding the spectral properties of the recurrent weight matrix W_{rec} . The necessary condition requires:

$$||\rho(W_{rec})|| < 1 \quad (2.19)$$

While the sufficient condition requires:

$$||\sigma_{max}(W_{rec})|| < 1 \quad (2.20)$$

Where $\rho(W_{rec})$ is the spectral radius of the matrix W_{rec} and $\sigma_{max}(W_{rec})$ is the largest singular value of the matrix W_{rec} , recall from [45][pp. 347], that the spectral radius is always upper bounded by the largest singular value: $\rho(W_{rec}) \leq \sigma_{max}(W_{rec}) = ||W_{rec}||_2$.

Although ESP guarantees that the reservoir state depends uniquely on the input history and not on initial conditions, enforcing its sufficient condition, $\rho(W_{rec}) < 1$, is often too restrictive in practice. This bound ensures that the recurrent dynamics are contractive even in the absence of input, thereby guaranteeing stability under all circumstances. However, it also leads to reservoirs with overly damped dynamics, where perturbations decay too quickly and the network loses its ability to store temporal information. The sufficient conditions is derived under the assumption of an autonomous system, neglecting the stabilizing effect of the continuous external input. Empirically, the most effective reservoirs operate near the edge of chaos and the spectral radius is often set even slightly above 1 in some cases [46].

2.6 Deep Recurrent Neural Networks

The concept of *depth* on RNNs is ambiguous and can be extended in multiple ways, as discussed by [47]. Firstly we say that a deep network applies multiple non-linear transformations to the input data, this is trivial for FNNs, but in the case of vanilla⁸ RNNs, the depth is embedded into the temporal structure, since we apply at each time step the same non-linearity. In the picture ?? we can see the that there are several ways to introduce depth in RNNs.

If we implement the depth from the *input-to-hidden* layers, the networks improves disentangling the underlying factors of variations from the original input signal, similar to how Convolutional layers in Convolutional Neural Networks (CNNs) extract features hierachically [something]. For *hidden-to-output* intermediate layers instead, the networks learn a more compact latent representation, with results in a more efficient summarization of input history into the hidden state that ù lead to the output layer, so the readout is simplified.

Yet another way to introduce depth is the *hidden-to-hidden* transition, which essentially allows the hidden state to rapidly adapt to mode shifts in the input signal, this

⁸In literature "vanilla" term refers to non-gated RNNs as the equation ?? describes

questa parte va
riscritta dopo

could be particularly useful in non-stationary environments, however a deep transition approach could lead to optimization issues due to the increased depth in the temporal dimension.

Finally we can stack multiple recurrent layers on top of each other, that is the main drive of this thesis, this approach encourages the model to learn at different timescales with the first layers capturing fast dynamics and the deeper layers capturing slower ones.

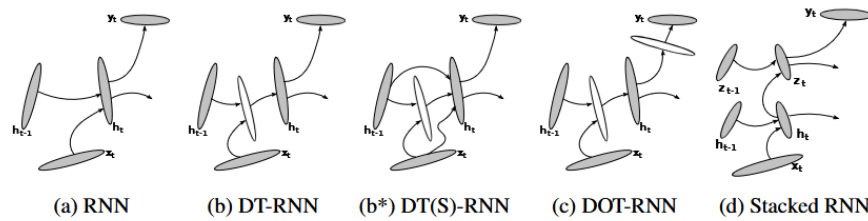


Figure 2: Illustrations of four different recurrent neural networks (RNN). (a) A conventional RNN. (b) Deep Transition (DT) RNN. (b*) DT-RNN with shortcut connections (c) Deep Transition, Deep Output (DOT) RNN. (d) Stacked RNN

FIGURE 2.4: Depths in Recurrent Neural Networks represented graphically by [47]

2.7 Deep Reservoir Computing

Chapter 3

Problem Statement and Related works

Chapter 4

Stacking RON Layers

Chapter 5

Implementation and Experimental results

Chapter 6

Conclusion and Future Works

Bibliography

- [1] Rajat Raina, Anand Madhavan, and Andrew Y. Ng. "Large-scale deep unsupervised learning using graphics processors". In: *Proceedings of the 26th Annual International Conference on Machine Learning*. ICML '09. Montreal, Quebec, Canada: Association for Computing Machinery, 2009, pp. 873–880. ISBN: 9781605585161. DOI: [10.1145/1553374.1553486](https://doi.org/10.1145/1553374.1553486). URL: <https://doi.org/10.1145/1553374.1553486>.
- [2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: [2006.11239](https://arxiv.org/abs/2006.11239) [cs.LG]. URL: <https://arxiv.org/abs/2006.11239>.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need*. 2023. arXiv: [1706.03762](https://arxiv.org/abs/1706.03762) [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- [4] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, et al. "Highly accurate protein structure prediction with AlphaFold". In: *nature* 596.7873 (2021), pp. 583–589.
- [5] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9 (Nov. 1997), pp. 1735–1780. DOI: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).
- [6] Rudolph Emil Kalman. "A new approach to linear filtering and prediction problems". In: (1960).
- [7] Leonard E Baum and Ted Petrie. "Statistical inference for probabilistic functions of finite state Markov chains". In: *The annals of mathematical statistics* 37.6 (1966), pp. 1554–1563.
- [8] Jeffrey L. Elman. "Finding structure in time". In: *Cognitive Science* 14.2 (1990), pp. 179–211. ISSN: 0364-0213. DOI: [https://doi.org/10.1016/0364-0213\(90\)90002-E](https://doi.org/10.1016/0364-0213(90)90002-E). URL: <https://www.sciencedirect.com/science/article/pii/036402139090002E>.
- [9] E.M. Izhikevich. "Simple model of spiking neurons". In: *IEEE Transactions on Neural Networks* 14.6 (2003), pp. 1569–1572. DOI: [10.1109/TNN.2003.820440](https://doi.org/10.1109/TNN.2003.820440).
- [10] Alan L Hodgkin and Andrew F Huxley. "A quantitative description of membrane current and its application to conduction and excitation in nerve". In: *The Journal of physiology* 117.4 (1952), p. 500.

- [11] Tom M Mitchell. *Machine learning*. Vol. 1. 9. McGraw-hill New York, 1997.
- [12] D. Barber. *Bayesian Reasoning and Machine Learning*. 04-2011. In press. Cambridge University Press, 2011. URL: <http://www.cs.ucl.ac.uk/staff/d.barber/brml>.
- [13] Vladimir N. Vapnik. *Statistical Learning Theory*. Wiley-Interscience, 1998.
- [14] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning*. 2nd. The MIT Press, 2018. ISBN: 0262039400.
- [15] Matus Telgarsky. *Deep learning theory lecture notes*. <https://mjt.cs.illinois.edu/dlt/>. Version: 2021-10-27 v0.0-e7150f2d (alpha). 2021.
- [16] Francis Bach. *Learning theory from first principles*. MIT press, 2024.
- [17] Jorge Nocedal and Stephen J. Wright. *Numerical optimization*. 2. ed. Springer series in operations research and financial engineering. New York, NY: Springer, 2006. XXII, 664. ISBN: 978-0-387-30303-1. URL: http://gso.gbv.de/DB=2.1/CMD?ACT=SRCHA&SRT=YOP&IKT=1016&TRM=ppn+502988711&sourceid=fbw_bibsonomy.
- [18] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [19] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: <https://arxiv.org/abs/1412.6980>.
- [20] Simon J.D. Prince. *Understanding Deep Learning*. MIT Press, 2023. URL: <http://udlbook.com>.
- [21] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314.
- [22] Kuniyiko Fukushima. “Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position”. In: *Biological cybernetics* 36.4 (1980), pp. 193–202.
- [23] Frank Rosenblatt. “The perceptron: a probabilistic model for information storage and organization in the brain.” In: *Psychological review* 65.6 (1958), p. 386.
- [24] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep learning”. In: *nature* 521.7553 (2015), pp. 436–444.
- [25] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *nature* 323.6088 (1986), pp. 533–536.
- [26] P. Tino, M. Cernansky, and L. Benuskova. “Markovian architectural bias of recurrent neural networks”. In: *IEEE Transactions on Neural Networks* 15.1 (2004), pp. 6–15. DOI: 10.1109/TNN.2003.820839.
- [27] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. *Sequence to Sequence Learning with Neural Networks*. 2014. arXiv: 1409.3215 [cs.CL]. URL: <https://arxiv.org/abs/1409.3215>.
- [28] Paul J Werbos. “Backpropagation through time: what it does and how to do it”. In: *Proceedings of the IEEE* 78.10 (2002), pp. 1550–1560.

- [29] Ken-ichi Funahashi and Yuichi Nakamura. "Approximation of dynamical systems by continuous time recurrent neural networks". In: *Neural networks* 6.6 (1993), pp. 801–806.
- [30] K. Doya. "Bifurcations in the learning of recurrent neural networks". In: *[Proceedings] 1992 IEEE International Symposium on Circuits and Systems*. Vol. 6. 1992, 2777–2780 vol.6. DOI: [10.1109/ISCAS.1992.230622](https://doi.org/10.1109/ISCAS.1992.230622).
- [31] Y. Bengio, P. Simard, and P. Frasconi. "Learning long-term dependencies with gradient descent is difficult". In: *IEEE Transactions on Neural Networks* 5.2 (1994), pp. 157–166. DOI: [10.1109/72.279181](https://doi.org/10.1109/72.279181).
- [32] Y. Bengio, P. Frasconi, and P. Simard. "The problem of learning long-term dependencies in recurrent networks". In: *IEEE International Conference on Neural Networks*. 1993, 1183–1188 vol.3. DOI: [10.1109/ICNN.1993.298725](https://doi.org/10.1109/ICNN.1993.298725).
- [33] Simon S. Haykin. *Neural networks and learning machines*. Third. Upper Saddle River, NJ: Pearson Education, 2009.
- [34] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. "On the difficulty of training recurrent neural networks". In: *Proceedings of the 30th International Conference on Machine Learning*. Ed. by Sanjoy Dasgupta and David McAllester. Vol. 28. Proceedings of Machine Learning Research 3. Atlanta, Georgia, USA: PMLR, 17–19 Jun 2013, pp. 1310–1318. URL: <https://proceedings.mlr.press/v28/pascanu13.html>.
- [35] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. *Deep Residual Learning for Image Recognition*. 2015. arXiv: [1512.03385](https://arxiv.org/abs/1512.03385) [cs.CV]. URL: <https://arxiv.org/abs/1512.03385>.
- [36] Quoc V. Le, Navdeep Jaitly, and Geoffrey E. Hinton. *A Simple Way to Initialize Recurrent Networks of Rectified Linear Units*. 2015. arXiv: [1504.00941](https://arxiv.org/abs/1504.00941) [cs.NE]. URL: <https://arxiv.org/abs/1504.00941>.
- [37] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. "Deep Sparse Rectifier Neural Networks". In: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. Ed. by Geoffrey Gordon, David Dunson, and Miroslav Dudík. Vol. 15. Proceedings of Machine Learning Research. Fort Lauderdale, FL, USA: PMLR, Nov. 2011, pp. 315–323. URL: <https://proceedings.mlr.press/v15/glorot11a.html>.
- [38] Herbert Jaeger. "The "echo state" approach to analysing and training recurrent neural networks-with an erratum note". In: *Bonn, Germany: German national research center for information technology gmd technical report* 148.34 (2001), p. 13.
- [39] Chrisantha Fernando and Sampsa Sojakka. "Pattern recognition in a bucket". In: *European conference on artificial life*. Springer. 2003, pp. 588–597.
- [40] Ronald J. Williams and David Zipser. "A Learning Algorithm for Continually Running Fully Recurrent Neural Networks". In: *Neural Computation* 1.2 (1989), pp. 270–280. DOI: [10.1162/neco.1989.1.2.270](https://doi.org/10.1162/neco.1989.1.2.270).

